Software Development Life Cycle (SDLC) Report

Cayeshni — Group Expense Management System

Seifeldin Ahmed Maazouz (8927)
Ahmed Wael Mohamed Gaber (8836)
Karim Mohamed Khaled (9004)
Abdelrahman Ahmed Agha (8918)
Abdelrahman Essam Eldin (8893)
Asser Mohamed Ahmed Hanafy (8833)

Faculty of Engineering, Alexandria University

May 13, 2026

Course: Software Engineering — final project (Term 08). Faculty of Engineering, Alexandria University. Document version 1.0. Submission date: May 13, 2026.

Contents

1 Cover and administrative data **1**

2 Abstract **1**

3 Introduction **1**
3.1 Why this report exists . . . 1
3.2 What the report covers . . . 1
3.3 What Cayeshni does (short) . . . 1

4 Software development life cycle **1**
4.1 Requirements . . . 2
4.1.1 Problem . . . 2
4.1.2 Scope . . . 2
4.1.3 Functional requirements (summary) . . . 2
4.1.4 Non-functional requirements (summary) . . . 3
4.1.5 Example scenario . . . 3
4.2 Design . . . 3
4.2.1 Architecture . . . 3
4.2.2 Main design choices . . . 3
4.2.3 Where the UML diagrams are . . . 3
4.3 Implementation . . . 4
4.3.1 Stack . . . 4
4.3.2 Main blocks of code . . . 4
4.3.3 Deployment . . . 4
4.4 Testing . . . 4
4.5 Maintenance . . . 4

5 Submission checklist **4**

6 References and annexes **4**
6.1 Primary reference . . . 4
6.2 Other internal files . . . 5
6.3 Annex — example scenario . . . 5

7 Document control **5**

1 Cover and administrative data

Copy these fields onto the official cover sheet for the PDF or printed submission.

Item	Entry
Title of project	Cayeshni — group expense management and settlement tracking
Course name	Software Engineering — final project (Term 08)
Instructor	As listed on the official course syllabus / LMS for your section
Team members	Seifeldin Ahmed Maazouz (8927); Ahmed Wael Mohamed Gaber (8836); Karim Mohamed Khaled (9004); Abdelrahman Ahmed Agha (8918); Abdelrahman Essam Eldin (8893); Asser Mohamed Ahmed Hanafy (8833).
Submission date	May 13, 2026
Institution	Faculty of Engineering, Alexandria University

Official brief for structure and deliverables: file `Final Project-software- SSP.pdf` in the `docs` folder.

2 Abstract

Cayeshni is a small web system for groups that share costs: one place to log who paid, how the bill was split, and who paid whom back later. This report follows the SDLC phases used in the module—requirements, design, implementation, testing, and maintenance—and points to the UML diagrams stored alongside the source (use case, sequence, activity, state, class). Diagrams live as separate files under `docs`; they are not inlined here so they stay easy to update.

3 Introduction

3.1 Why this report exists

The module asks for a written report that follows the life cycle, not only working code. This document states what we built, why, and how we checked it, without pasting source listings.

3.2 What the report covers

Everything here concerns the Cayeshni product. Behaviour and constraints are given in prose and tables; diagram sources are listed in Section 6.

3.3 What Cayeshni does (short)

Signed-in users can run groups (create, invite, join, leave), add expenses in the group currency with splits, see balances, and record settlements between two members when someone pays someone back. Friends are optional. The interface is a browser application; rules and data sit behind an HTTP API and a PostgreSQL database.

4 Software development life cycle

Each subsection below matches one phase from the course brief.

4.1 Requirements

4.1.1 Problem

Flatmates and trip groups often track money in chats or spreadsheets. That drifts out of date and people argue over the arithmetic. The aim is one shared ledger per group so the history of expenses and paybacks stays in one place.

4.1.2 Scope

In scope

ID	Item
S-01	Sign-up, login, logout, session refresh
S-02	Email confirmation before full access
S-03	Groups: create, join by invite, leave, owner edits
S-04	Friends: send / accept / decline, list friends
S-05	Group expenses with per-member split lines
S-06	Settlements between members, with checks on amounts and links to expenses where used

Out of scope

ID	Item
X-01	Card or bank payment APIs
X-02	Native iOS/Android apps (web only)
X-03	Live currency conversion beyond the group chosen currency

4.1.3 Functional requirements (summary)

ID	Text
FR-01	New users can register and sign in.
FR-02	Email must be confirmed before the account is treated as fully active.
FR-03	Users can create a group and share an invite so others can join.
FR-04	A member can post an expense: total plus splits that add up to that total.
FR-05	Group members can list expenses and open detail that shows how much is still owed after settlements.
FR-06	A settlement can be recorded between two members of the same group, with the rules enforced in code.
FR-07	Friend requests and acceptance so people can find each other before sharing groups.

4.1.4 Non-functional requirements (summary)

ID	Area	Text
NFR-01	Security	Passwords are not stored in plain text; access uses short-lived JWTs; refresh uses a protected cookie pattern; production should use HTTPS.
NFR-02	Performance	The app should stay usable for the group sizes used in class demos.
NFR-03	Usability	Screens are organised so a new user can complete the main flows without training.
NFR-04	Reliability	Inputs are validated; the database enforces keys and basic integrity.
NFR-05	Growth	The API is stateless so more instances could be added later if traffic grew.

4.1.5 Example scenario

A short vacation example used while writing requirements is in `docs/examples/vacation-trip-scenario.md` (see also Section 6.3).

4.2 Design

4.2.1 Architecture

Browser client, HTTP API, relational database. Identity and login use the platform identity support and JWTs on API calls.

4.2.2 Main design choices

The server is split into layers (HTTP, application services, domain, persistence) so each part can be tested and changed separately. Each group picks one default currency; all expenses and settlements for that group use it so balance math stays consistent.

4.2.3 Where the UML diagrams are

All paths below are relative to the `docs` folder.

Type	Files
Use case	`auth/auth-use-cases.md`, `groups/group-use-case-diagram.md`, `friends/friend-use-case-diagram.md`, `transactions/transaction-use-cases.md`, `settlements/settlement-use-cases.md`
Sequence	`auth/auth-sequence-diagrams.md`, `groups/group-sequence-diagrams.md`, `friends/friend-sequence-diagrams.md`, `transactions/transaction-sequence-diagram.` `settlements/settlement-sequence-diagram.md`
Activity	`design/activity-diagrams.md`
State machine	`design/state-machine-diagrams.md`
Class	`class_diagram.md`
Extra sketches	`auth/auth-architecture.md`, `groups/group-architecture.md`, `friends/friend-architecture.md`, `architecture/transaction-settlement-architectur`

The diagrams are deliberately small (few boxes and messages), closer to a pre-implementation sketch than a full reverse-engineering dump.

4.3 Implementation

4.3.1 Stack

Layer	Choice
UI	TypeScript, React, Next.js
API	C#, ASP.NET Core
Database	PostgreSQL via Entity Framework Core
Repository	Git; backend unit tests where added

4.3.2 Main blocks of code

Authentication and profiles; groups and membership; friends; transactions; settlements. Names align with folders in the backend solution.

4.3.3 Deployment

Configuration is read from environment settings (database URL, secrets, email). Schema changes go through EF migrations against the target database.

4.4 Testing

Unit tests cover pure logic and services; a smaller set of integration-style tests exercises the API where useful; manual runs cover sign-up through settlement in the browser. Sample checks:

Tag	Check
TC-01	Splits that do not equal the expense total are rejected.
TC-02	An expense that already has settlement lines tied to it cannot be deleted blindly.
TC-03	Someone who is not in the group cannot read that group transactions.

The tables can be extended if the instructor asks for a longer test appendix.

4.5 Maintenance

Day to day: keep the application running, apply security patches to dependencies and runtime, rotate secrets, back up the database according to institutional policy.

When bugs appear: reproduce, fix on a branch, review, merge—the same workflow as during development.

Future ideas (not part of this submission): receipt photos, CSV export, push or email reminders, a real payment provider.

5 Submission checklist

The brief asks for: (1) working software, (2) a project report as PDF, (3) the UML set. This repository contains (1) and the source text plus diagram files for (2) and (3). Before the deadline, compile this report to PDF, attach or embed diagrams as required, and submit according to course instructions.

6 References and annexes

6.1 Primary reference

Course handout: `docs/Final Project-software- SSP.pdf`.

6.2 Other internal files

List of diagram files: `docs/README.md`.
Branch naming and commits: `docs/CONTRIBUTING.md`.

6.3 Annex — example scenario

`docs/examples/vacation-trip-scenario.md`.

7 Document control

Ver.	Date	Who	Note
1.0	May 13, 2026	Authors (see §1)	Initial submission package (report + repository).

End of report. Export diagrams from the `docs` *tree into the final PDF where figures are required.*